

Latency-Budget Reference Card

Day 4 · Activity 3 handout. Take your latency SLO and walk it across the Container diagram. Each hop gets a budget; the sum must fit the SLO. If it doesn't, something has to give — surface that math *before* the feature ships and falls short.

Per-hop estimates (use these to annotate each arrow)

HOP TYPE	TYPICAL RANGE
Network hop (LAN → internet RTT)	20–80 ms
Database read (indexed)	5–30 ms
Database write (single-row)	10–50 ms
External API call (synchronous)	50–500 ms — wildly varies; look up real numbers
Async publish (Kafka / SQS, local)	1–10 ms

Async publishes are cheap on the hot path — but they don't count against user-visible latency the way a synchronous external call does. The **synchronous external call** is the hop teams most often underestimate.

The per-hop walk

If a feature has a top-level SLO of **p95 ≤ 400ms**, that 400ms must be **divided across the hops**:

```
[Mobile] →(80ms net)→ [API gateway] →(20ms)→ [Reconcile module]
                                         ↓
                                         (60ms) [Postgres read]
                                         ↓
                                         (20ms) [Tickets module call]
                                         ↓
                                         (40ms) [Postgres write]
                                         ↓
                                         (40ms) [Audit publish]
                                         ↓
Total: ~260ms server-side
+ 80ms net (round trip x2)
≈ 420ms
```

If the math doesn't fit — what gives

- **Tighter SLO target** (less ambitious).
- **Reduce hops** (cache, async, denormalize).
- **Faster individual hops** (engineering work).
- **Wider SLO percentile** (e.g., relax p99 → p95).

The **wider-percentile** option is the most overlooked. Don't reach for p99 if you have no instrumentation — p95 is usually fine for B2B SaaS.

Worked example — FieldPulse reconcile (SLO: p95 ≤ 1000ms)

```
Path: Mobile → API GW → Reconcile module → Postgres read
                                         → Tickets module
                                         → Postgres write
                                         → Audit publish
                                         → response back to Mobile

Mobile→API GW round trip:      80ms
API GW → Reconcile module:     20ms
Postgres read (current state): 30ms
Tickets module call (sync):    100ms (their median)
Postgres write:                40ms
Audit publish (async):         10ms
Reconcile → API GW response:   20ms
API GW → Mobile response:      80ms
-----
                               ~380ms median
                               p95 likely ~700-900ms
                               ✓ Fits the 1000ms SLO

Budget remaining for variance: ~100-300ms
Risk: Tickets module call dominates; if their latency degrades, we breach.
```

Your annotation table

ARROW / HOP	TYPE	ESTIMATE (MS)
Total (server + net round trip)		

- **SLO target:** _____ ms · **Sum vs SLO:** fits / doesn't fit
- **If it doesn't fit — what gives:** _____
- **Question for the architect:** _____